APOSTILA
do Prof. Eduardo

Sistemas Digitais e Microprocessadores

Arquitetura CISC

Prof. Eduardo F. Miranda
2023

CISC

Muitas instruções

Versatilidade

Complexidade (quando comparado com RISC, por ex.)

- Podem ser usados microprogramas internamente no processador para executar instruções mais complexas
  - Obs.: geralmente os programadores não têm acesso às microinstruções

Múltiplos modos de endereçamento

CISC nasceu em uma época que memórias eram caras e pequenas

- Compiladores deveriam gerar código compacto e eficiente
  - Instruções do processador deveriam executar várias funções

x86 CISC

O projeto inicial visava eficiência do software de alto nível

Conjunto de instruções para facilitar a construção de compiladores

Suporte de estruturas de alto nível

- Ex.: recursos para stack frames, end. simétrico, arrays, e strings

Conforme foi evoluindo, novas instruções foram sendo incorporadas

- Compatibilidade com software antigo

Nas microarquiteturas P6 (Pentium Pro) e posteriores, as instruções x86 são convertidas internamente em microoperações mais simples no estilo RISC

Desenvolvimento da arquitetura CISC

3 aspectos

- Utilização de micro-código
- Instruções completas e eficientes
- Instruções complexas auxiliando compiladores e ling. de alto nível

Regras usualmente seguidas no desenvolv. da arquitetura CISC

- Instruções com campos de origem e destino
  - Ex.: sem precisar de instruções adicionais para para ler/gravar entre memória e registradores
- 3 modos de transferência de dados
  - Registrador para registrador
  - Registrador para memória
  - Memória para registrador

(continua)

(continuação)

- Diferentes modos de endereçamento para a memória
- Instruções com largura variável
  - Qtde. de bytes variando de acordo com o modo de endereçamento
- Instruções executadas em múltiplos ciclos de clock
  - Varia conforme a largura da instrução
- Processador possui poucos registradores
  - Existência de várias instruções com acesso à memória
- Existência de registradores especializados
  - Controle, segmento, ponteiros de pilha, etc.

CISC é anterior à RISC

- RISC procurou resolver alguns problemas que aconteciam com a CISC
- Ex.: grande quantidade de instruções
  - Acabavam prejudicando o desempenho global

Pontos problemáticos da CISC

Diferenças de velocidade entre memória e processador

Microcódigo

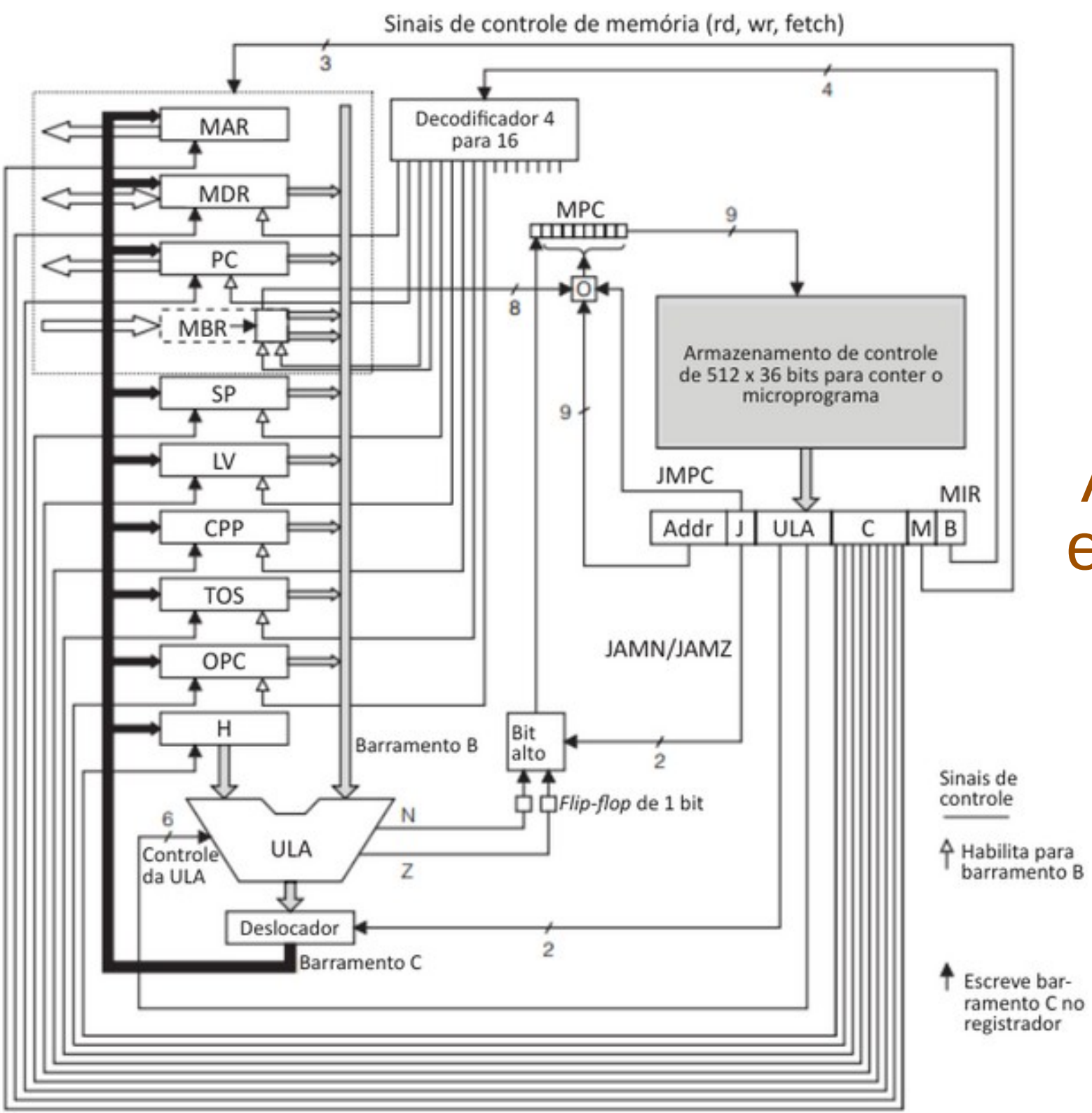
- No final o RISC abava sendo usado de qualquer forma
- Por que não simplificar e usar o RISC direto?

Grande quantidade de instruções

- Apesar do suporte a compiladores e linguagens de alto nível, aumenta a complexidade e diminui o desempenho global
- Apesar do código parecer mais denso e com poucas instruções, ao executar microinstruções, por exemplo, a quantidade total de passos de execução acaba sendo grande

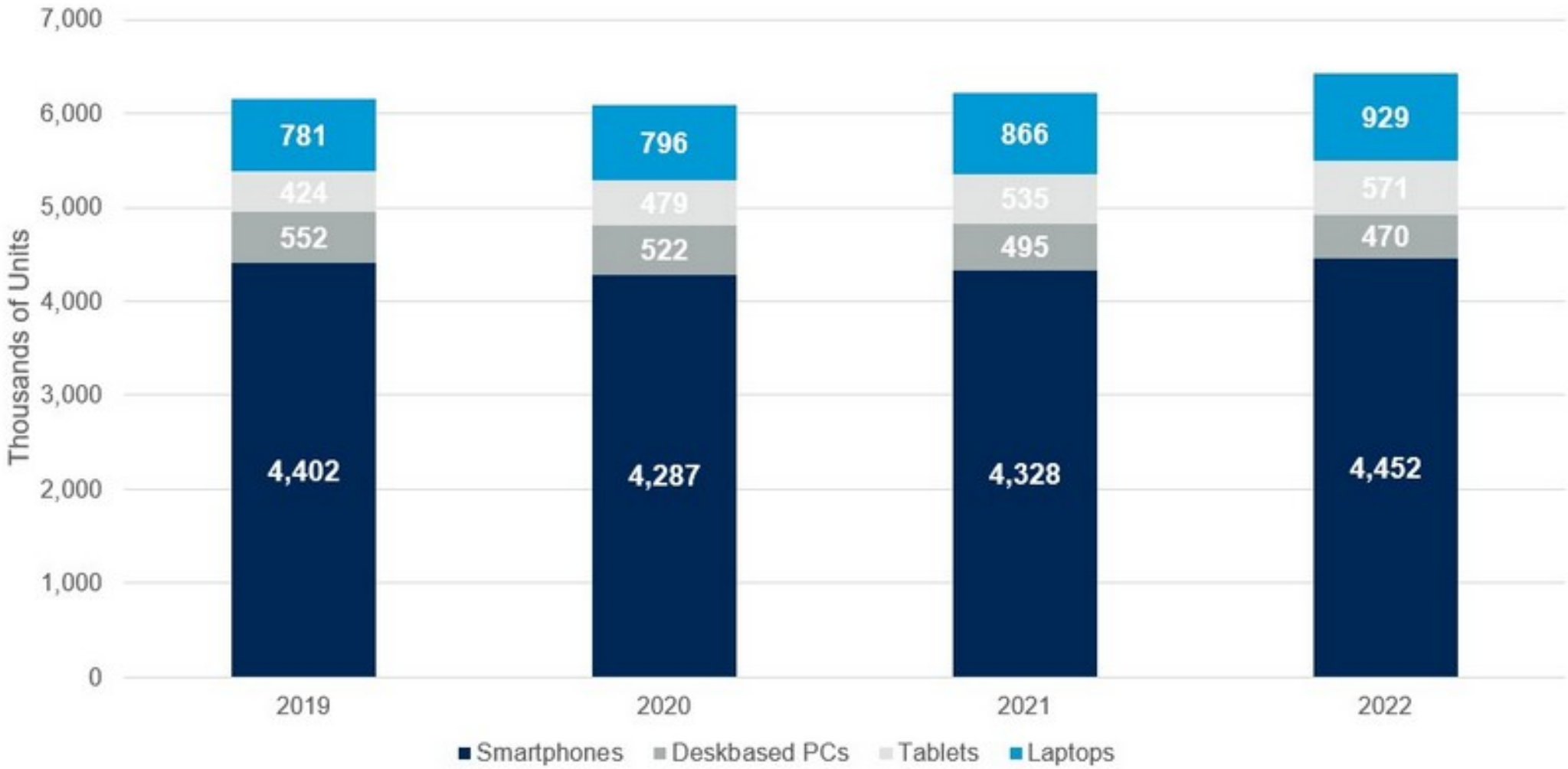
Necessidade de compatibilidade com processadores anteriores

- RISC por ser mais simples, não tem esse problema tão acentuado



Arquitetura educacional Mic-1

RISC vs CISC



https://tiinside.com.br/22/04/2021/base-instalada-de-dispositivos-chegara-a-62-bilhoes-este-ano-estima-gartner/

Características	RISC	CISC
Complexidade e quantidade de instruções	Instruções simples e em número reduzido.	Muitas instruções complexas.
Tamanho das instruções	Instruções de tamanho fixo.	Instruções de tamanho variável.
Complexidade	Complexidade no compilador.	Complexidade no código.
Modos de endereçamento	Poucos modos de endereçamento.	Muitos modos de endereçamento.
Quantidade de ciclos	Instruções em um único ciclo de máquina.	Instruções em múltiplos ciclos de máquina.
Paralelismo	Altamente paralelizado.	Fracamente paralelizado.
Acesso aos dados	Acesso aos dados via registradores.	Acesso aos dados via memória.
Microprogramação	Execução sem microprogramação.	Execução com microprogramação.
Quantidade de registradores	Vários registradores.	Poucos registradores.

RISC

Unidades de controle mais simples, rápidas, baratas

Instruções mais simples, pouca variedade, poucos endereços

Consequentemente os circuitos internos são mais simples

Geram pouco calor, consomem relativamente pouca energia

- O projeto do sistema também prevê pouco consumo

Como as instruções variam pouco, a unidade de controle consegue prever melhor quantos ciclos de máquina serão necessários

- Ganho de desempenho no processamento paralelizado

Geralmente são menores, mais baratos, consomem menos

- A aplicação em aparelhos móveis influencia o desenvolvimento

CISC

Geralmente utilizado em máquinas de maior desempenho

- Embora existam supercomputadores que utilizam RISC

Grande quantidade de instruções

Ênfase na construção de compiladores e linguagens de alto nível

Maior densidade de código

Geralmente são máquinas maiores, consumindo mais energia, e com maior capacidade de processamento

Geralmente associado à arquitetura von Neumann

- Embora internamente o processador possa utilizar a arquitetura Harvard
  - Ex.: memória cache separada para dados e instruções

RISC x CISC

Multiplicação a = a * b, com a no end1 e b no end2

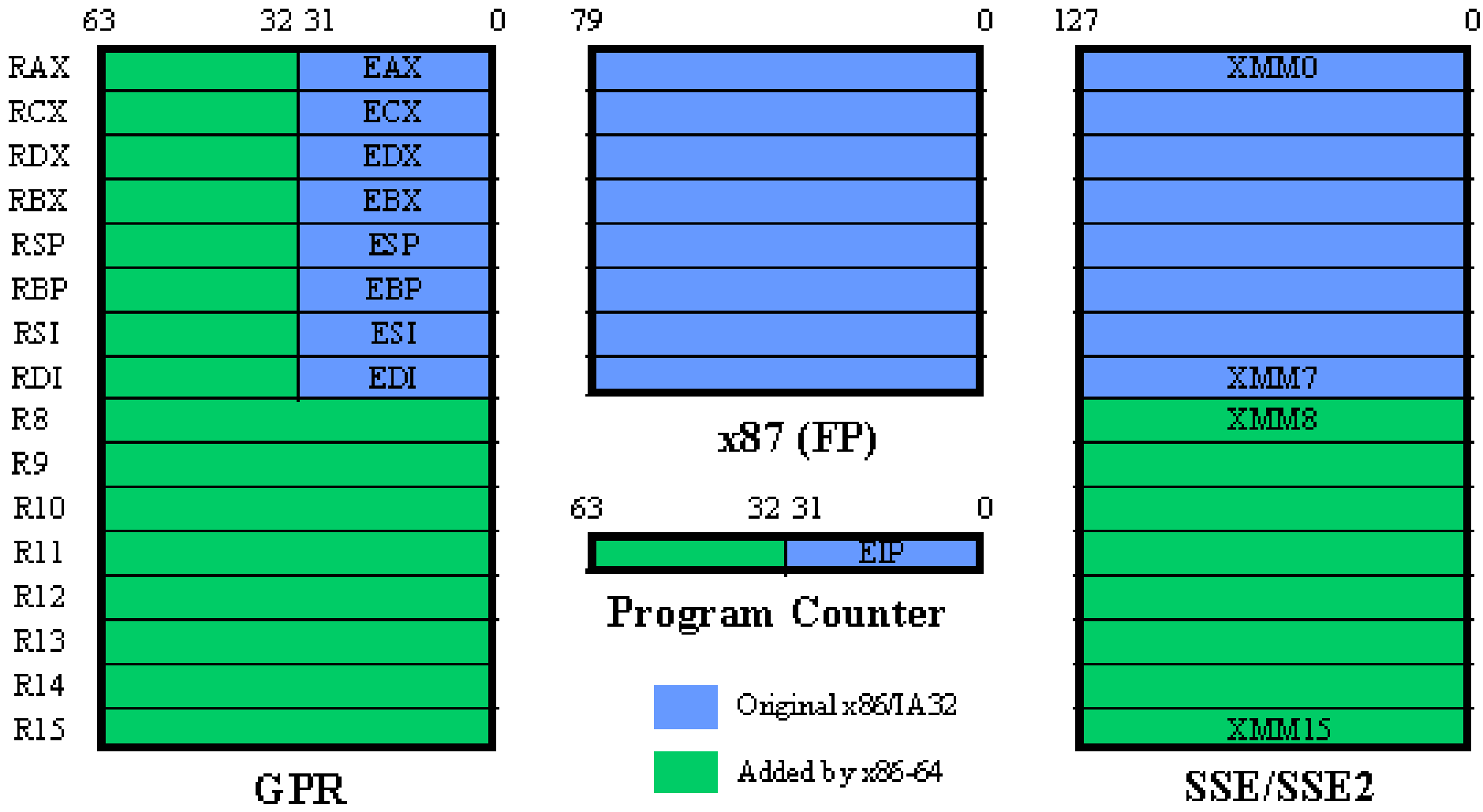
CISC

```
MULT end1, end2
```

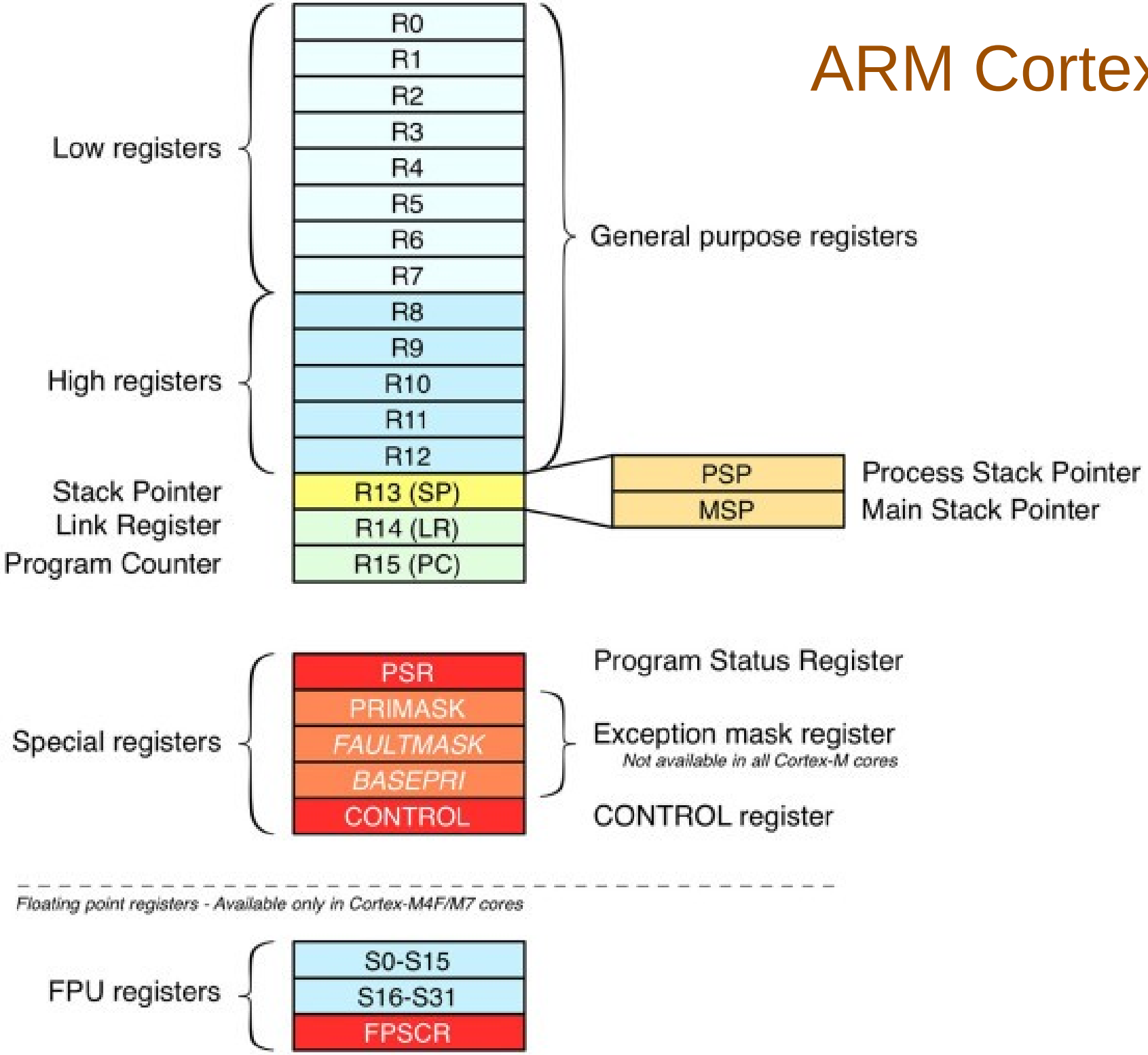
RISC

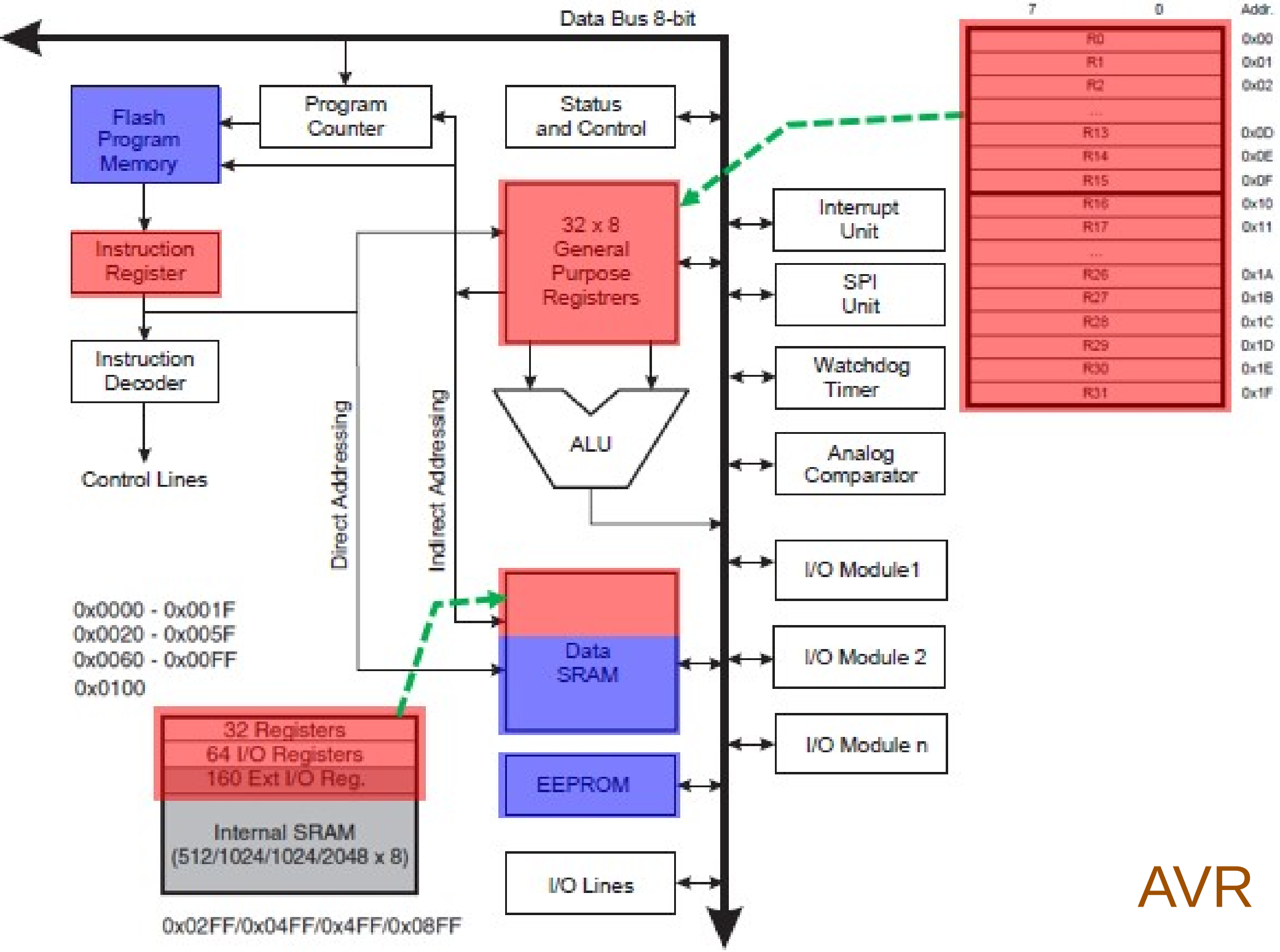
```
LOAD R1, end1
LOAD R2, end2
PROD R1, R2
STORE end1, R1
```

x86 64


63
32 31
0
RAX
RCX
RDX
RBX
RSP
RBP
RSI
RDI
R8
R9
R10
R11
R12
R13
R14
R15
EAX
ECX
EDX
EBX
ESP
EBP
ESI
EDI
GPR
79
0
x87 (FP)
63
32 31
0
EIP
Program Counter
Original x86/IA32
Added by x86-64
127
0
XMM0
XMM7
XMM8
XMM15
SSE/SSE2

ARM Cortex M4




Data Bus 8-bit
Flash Program Memory
Program Counter
Status and Control
Instruction Register
Instruction Decoder
Control Lines
32 x 8 General Purpose Registrers
ALU
Direct Addressing
Indirect Addressing
Interrupt Unit
SPI Unit
Watchdog Timer
Analog Comparator
I/O Module1
I/O Module 2
I/O Module n
Data SRAM
EEPROM
I/O Lines
0x0000 - 0x001F
0x0020 - 0x005F
0x0060 - 0x00FF
0x0100
32 Registers
64 I/O Registers
160 Ext I/O Reg.
Internal SRAM
(512/1024/1024/2048 x 8)
0x02FF/0x04FF/0x4FF/0x08FF
7
0
Addr.
R0
R1
R2
...
R13
R14
R15
R16
R17
...
R26
R27
R28
R29
R30
R31
0x00
0x01
0x02
0x0D
0x0E
0x0F
0x10
0x11
0x1A
0x1B
0x1C
0x1D
0x1E
0x1F

AVR